

Asymmetric Cryptography

Part 1

Computer Systems Security

2025/26

LEI - UM

Hugo Pacheco hpacheco@di.uminho.pt

Symmetric Cryptography

Key distribution

- Symmetric cryptography is more efficient and less complex
- Its major challenge is the pre-distribution of keys, **notice the circularity:**

Symmetric cryptography can be used to establish a secure channel between two parties, but itself depends on the existence of a secure channel to distribute the secret key

- Using short-term keys as recommended makes the problem even worse
- Asymmetric cryptography will offer more attractive solutions
 - At the cost of worse performance and stronger assumptions

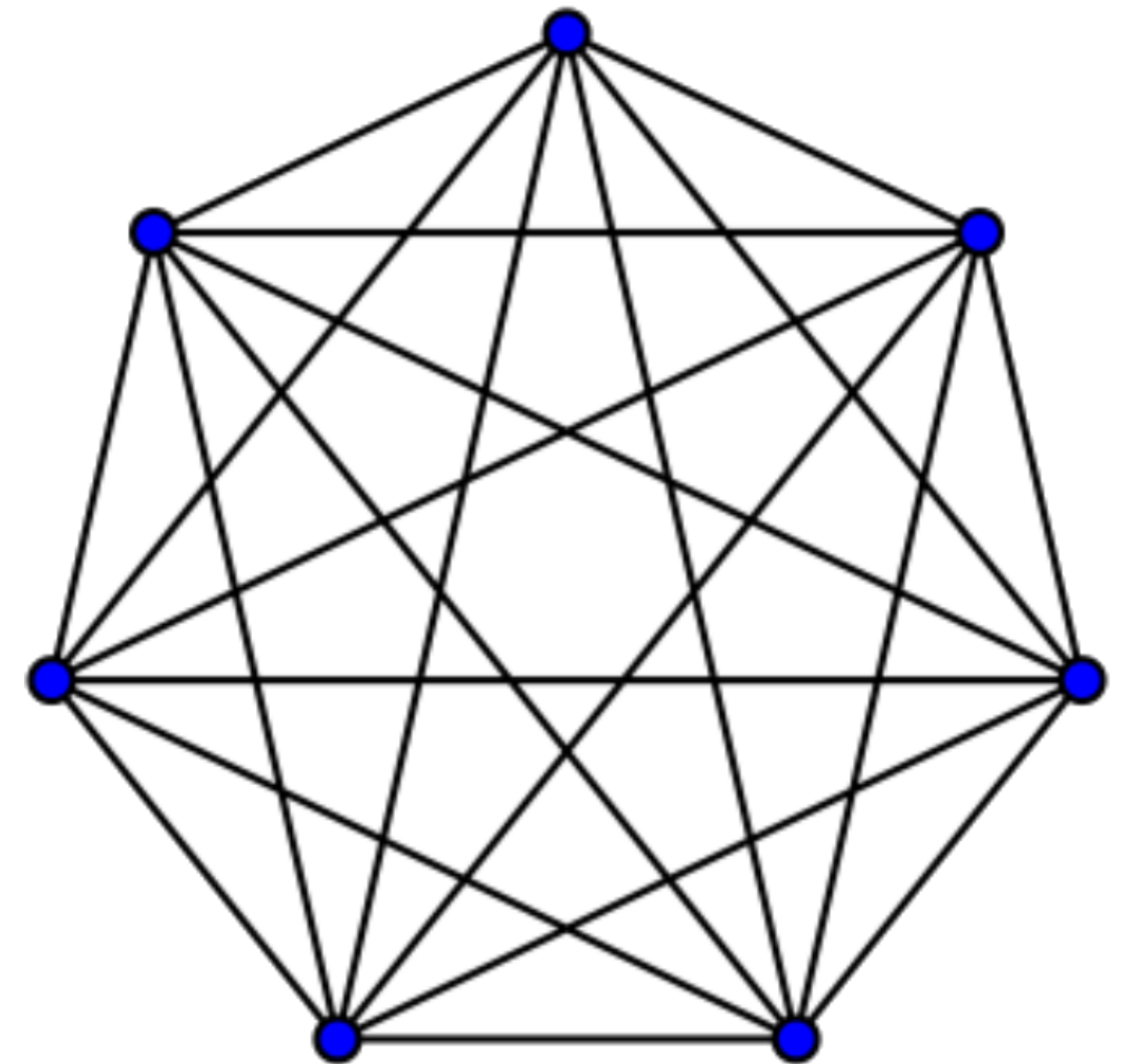
Key Distribution

Limitations

- For N users, the total number of keys necessary for secure communication between any pair is

$$\binom{N}{2} = N(N - 1) / 2$$

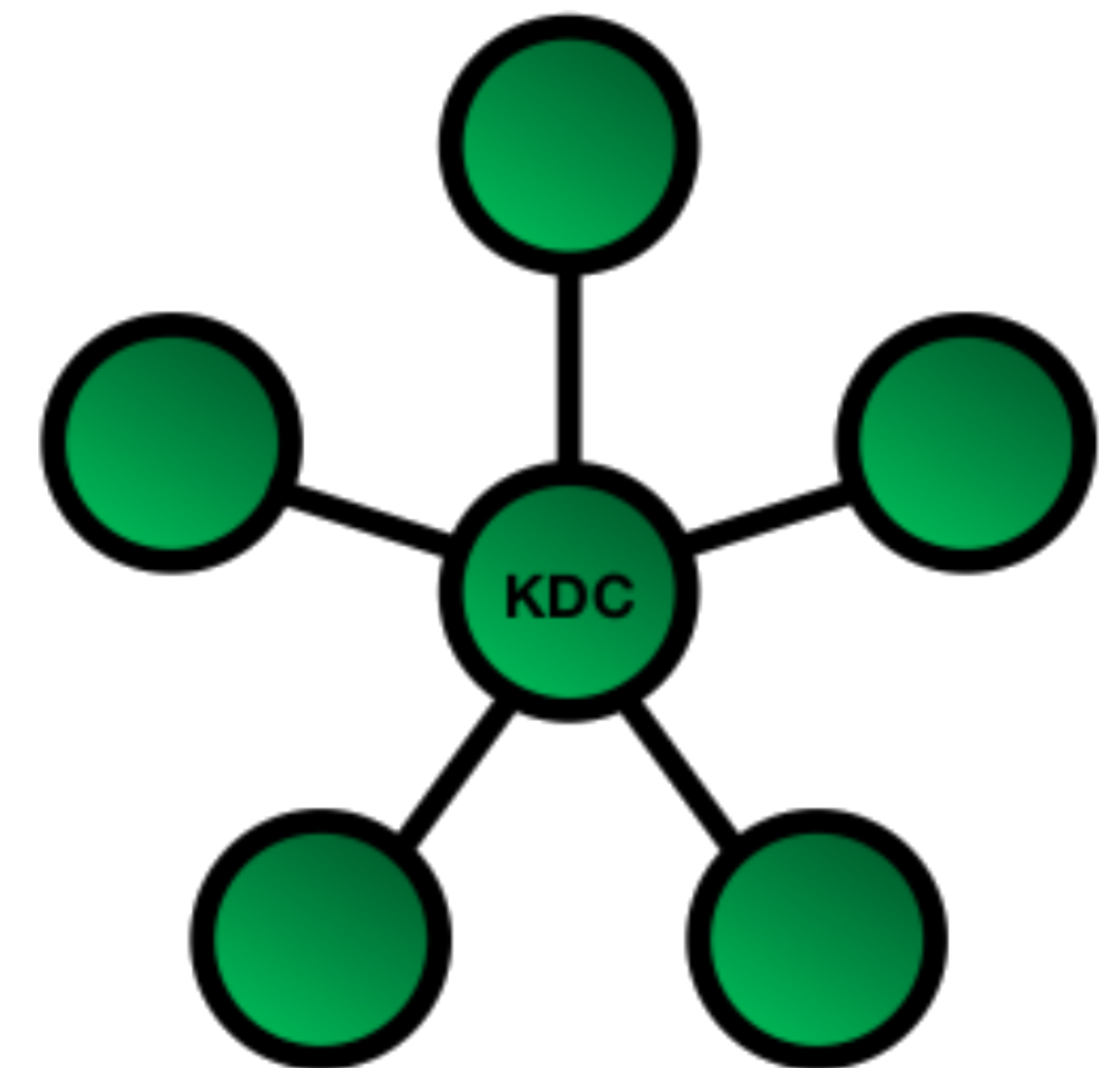
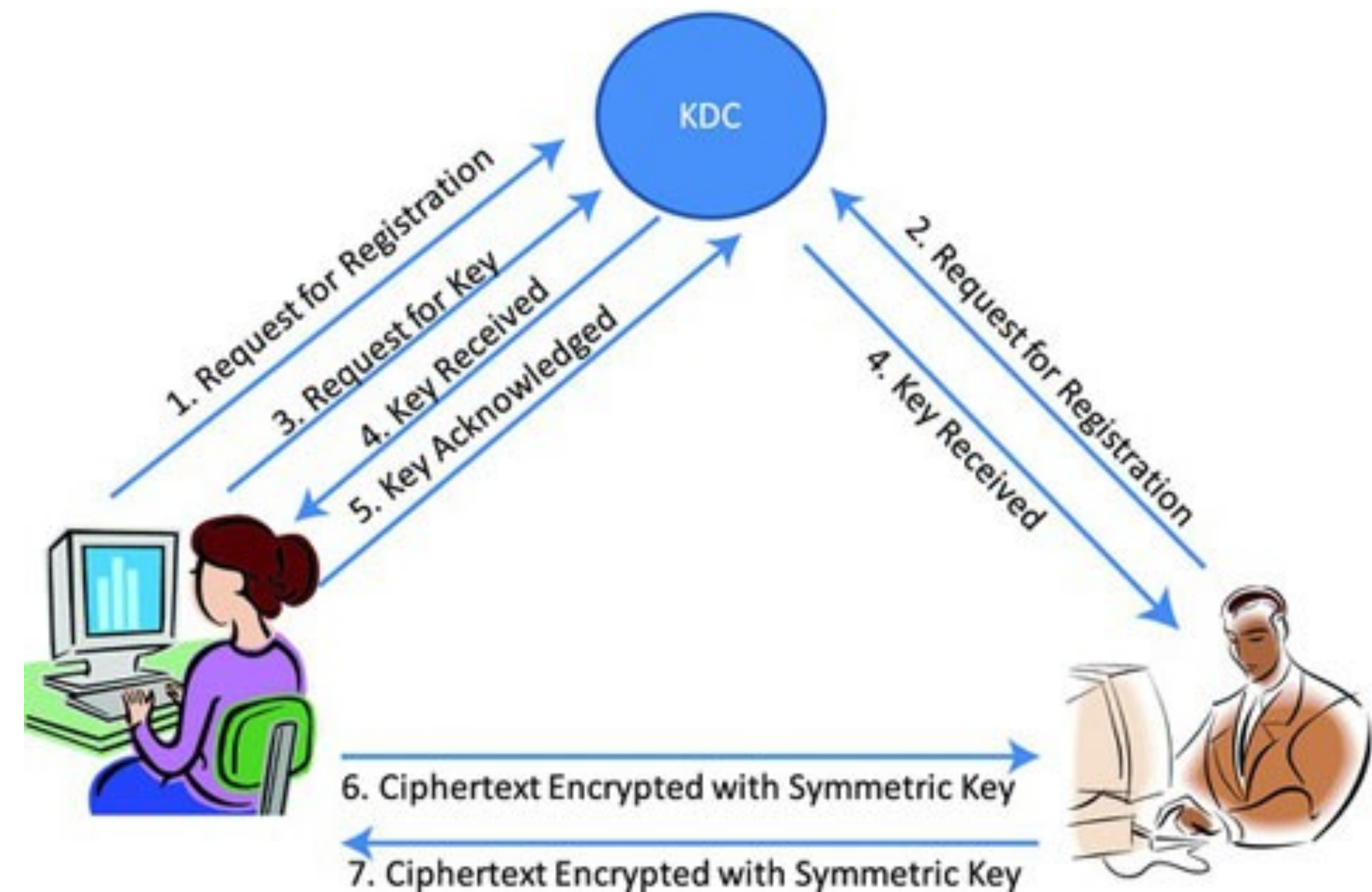
- This is clearly not feasible in a community of considerable size (e.g. Internet)
- Are keys to be pre-distributed? Manually?
- What if participants can change dynamically?



Key Distribution

Closed systems

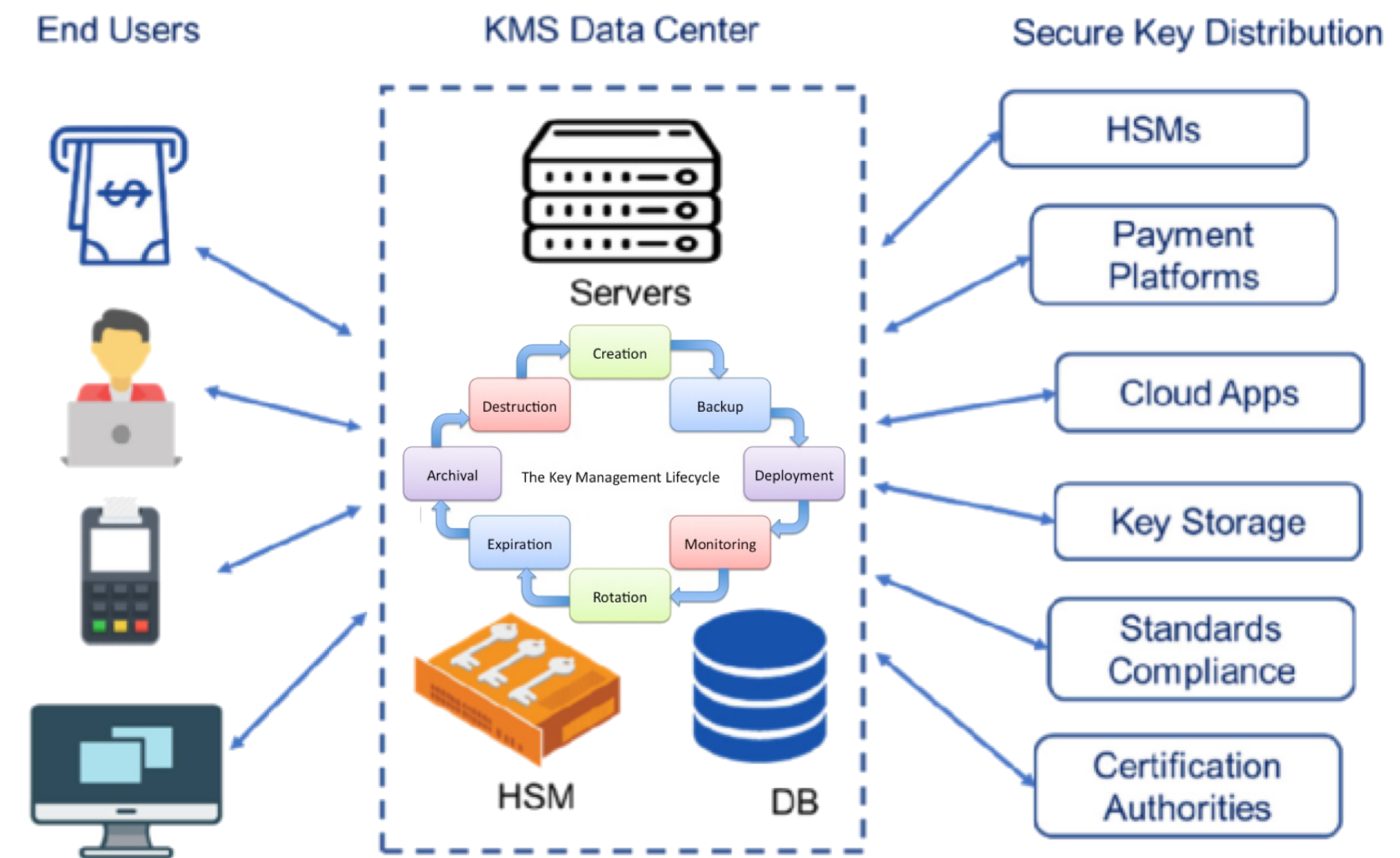
- We can consider a **Key Distribution Protocol**
 - Key Distribution Center (KDC), e.g. Kerberos
 - stores 1 long-term key for each of N users
 - Shares long-term keys with each user
 - Generates and distributes a session-key between any pair of users on demand
 - **KDC is a central point of failure**
 - Trusted Third-Party (TTP) for all users
 - Always on-line
- Suitable for small communities (e.g., LAN), but not for open communities (e.g., Internet)



Key Distribution

Cryptographic Key-Management Systems (CKMS)

- **Key lifecycle:** covers all states that a key passes between its generation and its destruction
- **Security Policies:** key lengths, rotation intervals, and access control
- **HSMs:** physical devices that provide a “Root of Trust”
- Modern trends towards Decentralized KMS (DKMS)
 - **Multi-cloud** for scalability (AWS, Azure, Google Cloud)
 - **Blockchains** for tamper-proof public metadata and security policies
 - **MPC** for key storage without a single point of trust



Key Agreement Revolution in the 70s

- Are there viable alternatives to pre-agreement / distribution of keys?
- What if both parties agree on a common secret...
 - ...exchanging messages on a public channel...
 - ...but without it being possible to derive the secret knowing only the messages exchanged
- Such a scheme first appeared in the article “New Directions in Cryptography” (Diffie & Hellman 1976)
 - Security relies on hard mathematical problems from number theory

New Directions in Cryptography

Invited Paper

WHITFIELD DIFFIE AND MARTIN E. HELLMAN, MEMBER, IEEE

WE STAND TODAY on the brink of a revolution in cryptography. The development of cheap digital hardware has freed it from the design limitations of mechanical computing and brought the cost of high grade cryptographic devices down to where they can be used in such commercial applications as remote cash dispensers and computer terminals. In turn, such applications create a need for new types of cryptographic systems which minimize the necessity of secure key distribution channels and supply the equivalent of a written signature. At the same time, theoretical developments in information theory and computer science show promise of providing provably secure cryptosystems, changing this ancient art into a science.

Key Agreement

Three-pass protocol

- Alice wants to send a message to Bob with **no pre-agreed shared key**

- Alice holds a key A , Bob holds a key B

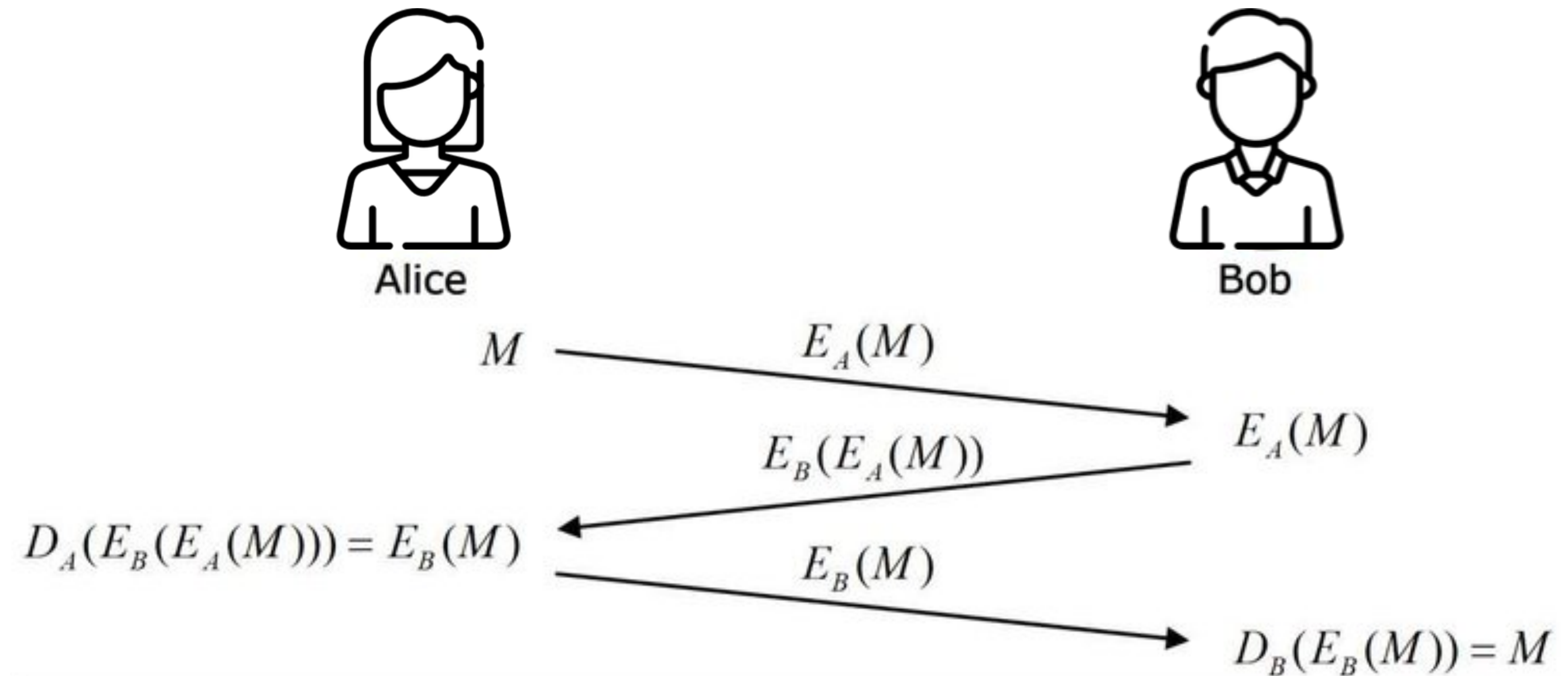
- They use a symmetric cipher

- Notice that:

- Alice and Bob do not share their respective secret keys

- Message M is always protected with at least one layer of encryption

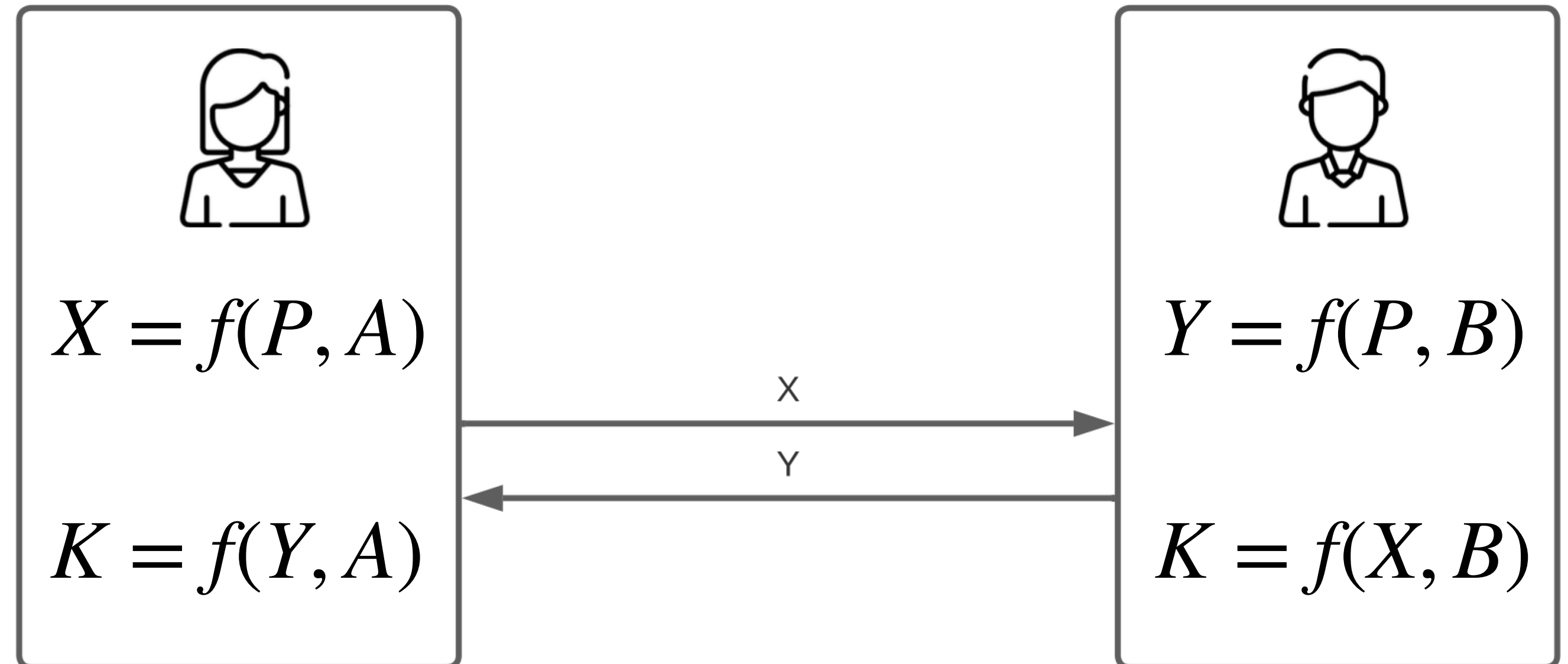
- Works as long as encryption is commutative: $E_A(E_B(M)) = E_B(E_A(M))$



Key Agreement

Diffie-Hellman

- Alice and Bob want to **agree on a shared key**
- Alice holds a key A
- Bob holds a key B
- Assume:



- A special (two-input) one-way function f (where P is a public parameter)
 - One-wayness formulated as: given x and $f(x, y)$, it is hard to find y
- Works as long as the one-way function is commutative: $f(f(P, A), B) = f(f(P, B), A)$

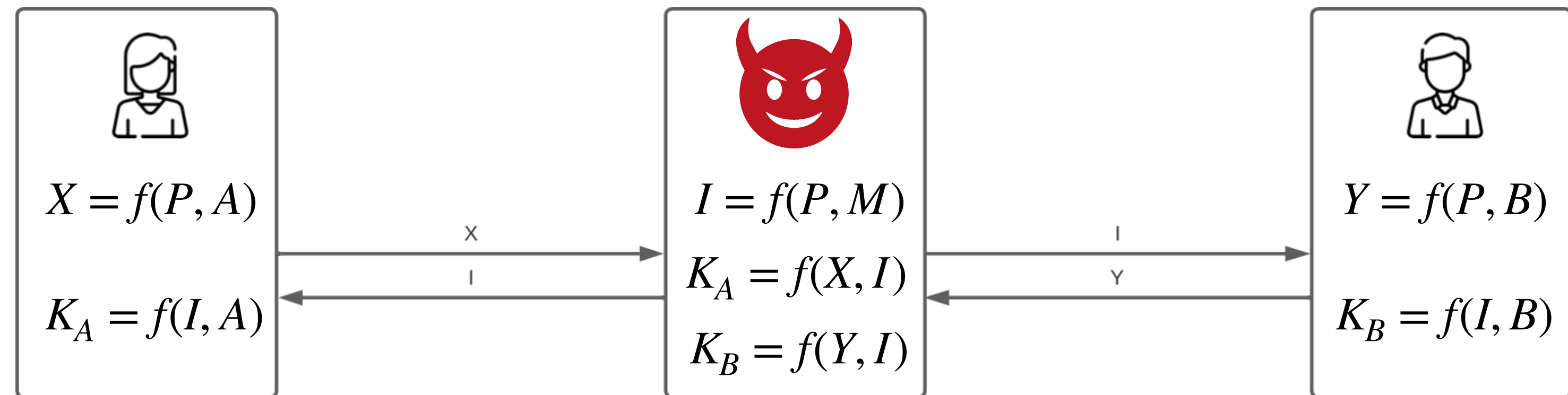
Key Agreement

Diffie-Hellman

- Much better to agree on a key compared to sending a message
 - **Alice and Bob can now use an efficient symmetric cipher**

- Preserves anonymity
 - Alice and Bob might not be sure who the other is

- **But vulnerable to active man-in-the-middle attacks**



- If attacker simply forwards messages, it's undetectable

Intermezzo

Algebraic Structures

- A **group** (G, op) is an algebraic structure where G is a set and op a total binary operation
 - Its **order** denotes the number of elements in G
 - It is called **abelian** if op is commutative
 - E.g., the abelian group of integers under addition $(\mathbb{Z}, +)$
- A **ring** (R, op_1, op_2) is an abelian group such that op_2 is associative and distributive over op_1
 - E.g., the ring of integers $(\mathbb{Z}, +, \cdot)$
- A **field** (F, op_1, op_2) is a ring such that each element has a multiplicative inverse
 - E.g., the field of real numbers $(\mathbb{R}, +, \cdot)$

Intermezzo

Algebraic Structures

- For composite n , $(\mathbb{Z}_n, +, \cdot)$ forms a **ring** ($+$ and $\cdot$ are respectively addition and multiplication modulo n)
 - Elements with multiplicative inverse (units) are those x such that $\gcd(x, n) = 1$ (primes relative to n)
 - The number of those elements are given by $\phi(n)$ (Euler function)
- When p is a prime number, $GF(p) = (\mathbb{Z}_p, +, \cdot)$ is a **field**
 - Modular inverses can be computed by the (extended) Euclidean algorithm
 - A primitive element is a **generator** g of the multiplicative group (that is, each non-zero element can be written as g^i , for some i)

Intermezzo

(Believed) Hard problems

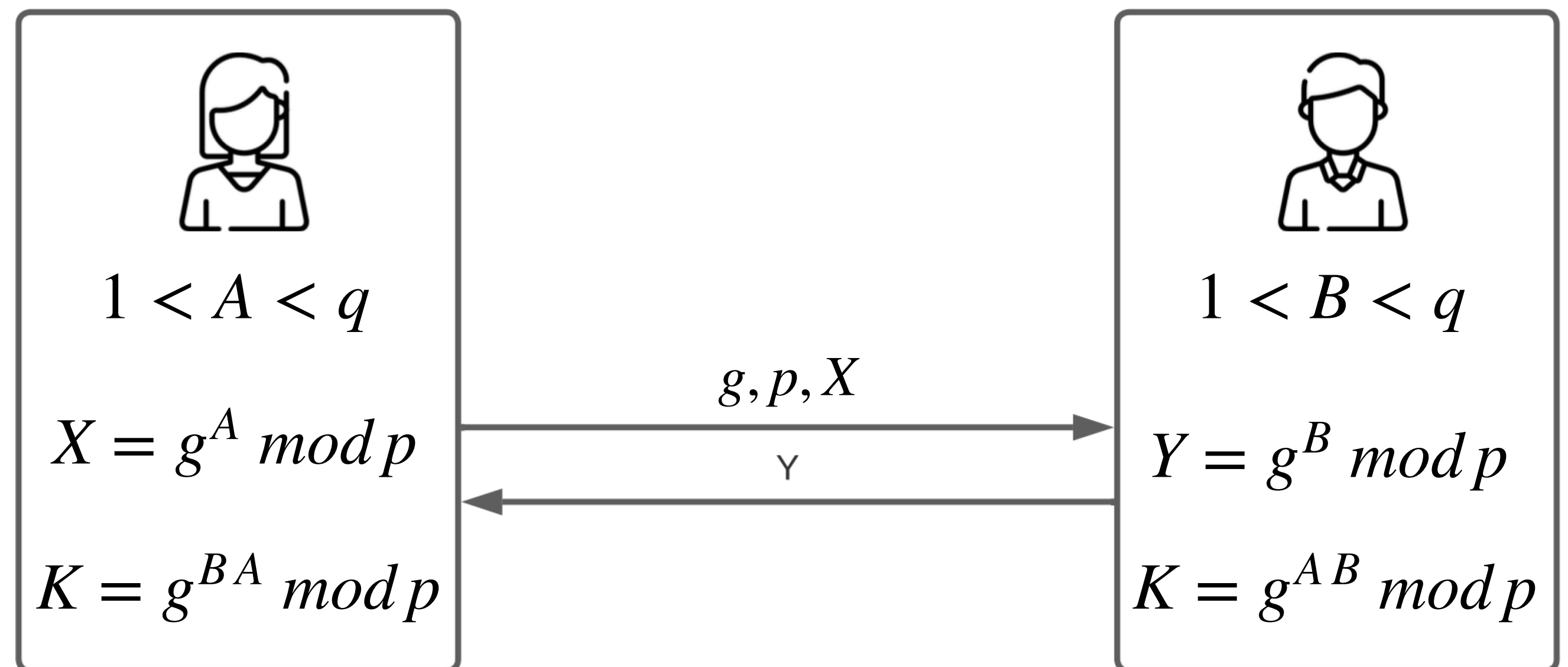
Under appropriate conditions, the following problems are considered difficult

- **Integer factorisation**
 - Given an integer n , determine its prime number factorisation. In other words, determine the prime numbers $p_1, \dots, p_i$ such that $p_1 \cdot \dots \cdot p_i = n$
- **Discrete square-root**
 - Given y and n , compute x such that $x^2 \bmod n = y$
- **Discrete logarithm**
 - Given a, b and n , compute x such that $a^x \bmod n = b$

Key Agreement

Diffie-Hellman (math)

- Let p be a prime and g a generator of a subgroup of $(\mathbb{Z}_p, \cdot)$ of prime order q
- We can define the one-way function as $f(y) = g^y \bmod p$
- Commutativity holds: $(g^x)^y = g^{xy} = g^{yx} = (g^y)^x \bmod p$
- Computational security follows from one-wayness of modular exponentiation (hardness of discrete logarithm for a PPT adversary)



Key Agreement

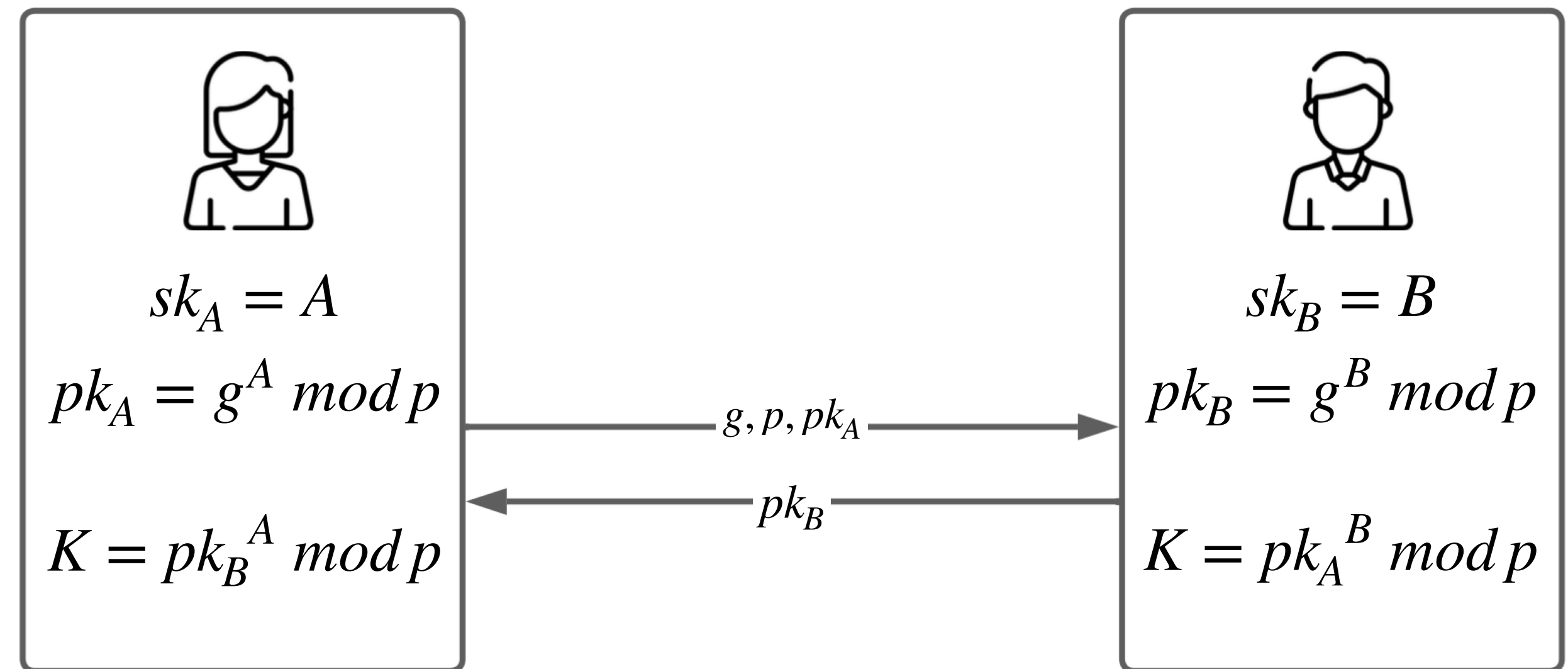
Diffie-Hellman (applications)



- Still used today, the technique is the cornerstone of asymmetric cryptography
- Crucial for applications such as HTTPS
 - Decades until we could converge to a secure and efficient solution
 - Designing a complete protocol is extremely complex
 - Composing several primitives, implementations, etc
 - TLS 1.3 is the result of 30 years of evolution
- **Forward secrecy:** ephemeral session keys generated for each execution
- **Authentication:** can be combined with digital signatures (later) to prevent MitM attacks

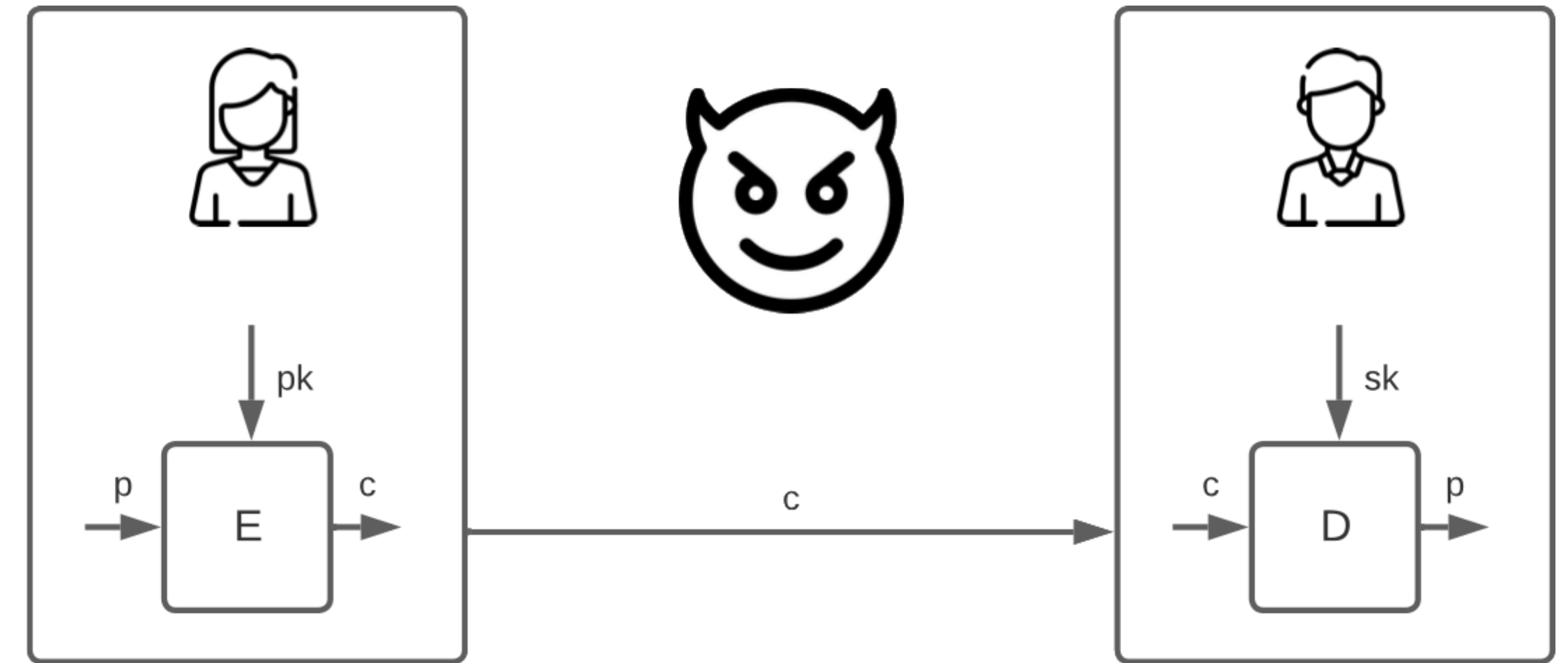
Public-Key Cryptography

- The DH article suggested other possible applications of the underlying mechanism: **splitting the key into two components (a key pair)**
 - A **secret key (sk)** that remains secret
 - A **public key (pk)** that can be revealed
- This is latent in the original design



Public-Key Encryption

- We can extrapolate what this means for **Public-Key Encryption (PKE)**
 - Bob generates a key pair (sk_B, pk_B)
 - Bob sends pk_B to Alice
 - Alice encrypts a message p with Bob's public key and sends $c = E(pk_B, p)$
 - Bob decrypts c using his private key $p = D(sk_B, c)$
- We need an asymmetric encryption scheme such that $D(sk, E(pk, m)) = m$



Public-Key Encryption

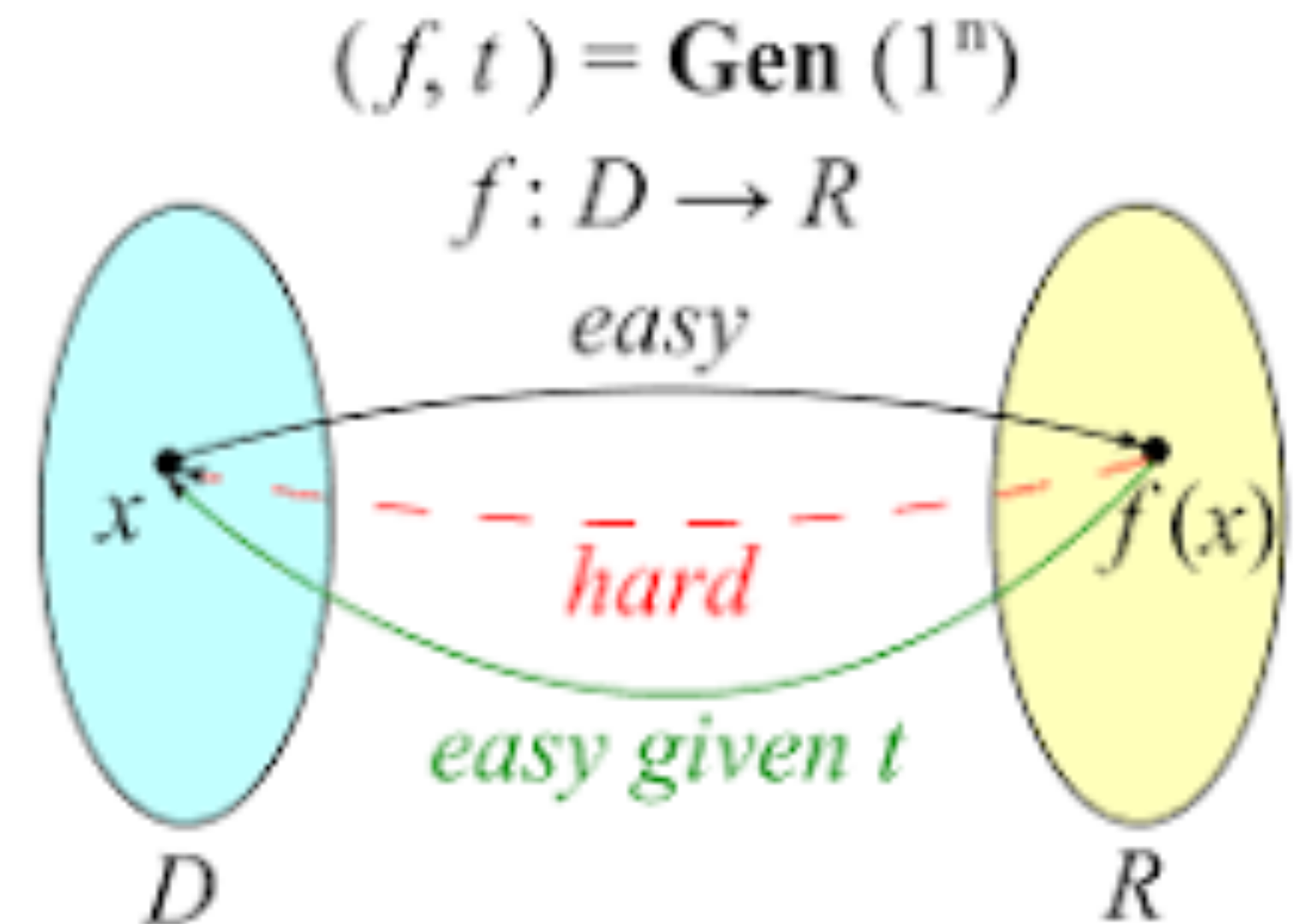
Intuition

- **Anyone with the public key can encrypt**
 - Public keys can be pre-shared in some some public repository
- **Only the holder of the secret key can decrypt**
 - At any time! The protocol is non-interactive and parties no longer need to be simultaneously online
- **Key pairs represent long-term keys**
 - Unlike the short-term session keys for secure channels under symmetric encryption

Public-Key Encryption

Trapdoor one-way permutations

- We need a primitive known as a **trapdoor one-way permutation**
- Compared to a simple OWF (as used in DH):
 - **Permutation:** the function must now **be invertible**, such that it is possible to recover the input
 - Note that the co-domain can no longer be significantly smaller than the domain (as used to be the case for a cryptographic hash function)
 - **Trapdoor:** the function must still be **hard to invert (one-way) in general**, but **easy to invert given a special “trapdoor” value t**



Public-Key Encryption

ElGamal

- PKE algorithm introduced in 1984 by *Taher El Gamal*
- An extension of Diffie-Hellman
 - Let p be a prime and g a generator of a subgroup of $(\mathbb{Z}_p, \cdot)$ of prime order q
 - $\text{KeyGen}(p, g) = (sk, pk)$ where $sk = x$ and $pk = g^x$ for random x such that $1 < x < q$
 - $\text{Enc}(pk, m) = (g^k \bmod p, m \cdot pk^k \bmod p)$ for a random “one-time” mask k such that $1 < k < q$
 - $\text{Dec}(sk, (c_1, c_2)) = c_2 / c_1^{sk} \bmod p$
- **Correctness:** $\text{Dec}(sk, \text{Enc}(pk, m)) = \text{Dec}(x, (g^k, m \cdot g^{xk})) = (m \cdot g^{xk}) / g^{kx} = m \bmod p$
- **Computational security:** discrete logarithm problem + k being random and ephemeral

Public-Key Encryption

ElGamal (properties)

- Note that:
 - Encryption is a randomised algorithm
 - Requires a new ephemeral mask per message encryption
 - Ciphertext doubles the size of the plaintext
- A scheme with great academic relevance but little practical impact
 - Doubling the size is very costly (especially when PK keys are already quite large)
 - Security depends on mask being “one-time” and fully random

Public-Key Encryption

ElGamal (properties)

- An enigmatic cryptographic construction is **homomorphic encryption**
 - Being able to perform computations on encrypted data
 - Very powerful and with applications in e-voting, etc
- As an example, ElGamal encryption is multiplicatively homomorphic

$$\text{Enc}(pk, m_1) \cdot \text{Enc}(pk, m_2) = \text{Enc}(pk, m_1 \cdot m_2)$$

$$m_1 \cdot pk^{k_1} \cdot m_2 \cdot pk^{k_2} = (m_1 \cdot m_2) \cdot (pk^{k_1} \cdot pk^{k_2}) = (m_1 \cdot m_2) \cdot pk^{k_1 k_2}$$

(Can be made additively homomorphic, but not both simultaneously - fully homomorphic)

Public-Key Encryption

RSA

- The first PKE algorithm, proposed by *Ron Rivest, Adi Shamir & Leonard Adleman* (1977/8)
 - Let p and q be two primes and e a public exponent such that $n = p \cdot q$ and $\gcd(e, n) = 1$
 - $\text{KeyGen}(n, e) = (sk, pk)$ where $sk = (n, d)$ and $pk = (n, e)$
 - Compute d such that $e \cdot d = 1 \bmod \phi(n)$, where $\phi(n) = (p - 1) \cdot (q - 1)$
 - $\text{Enc}((n, e), m) = m^e \bmod n$
 - $\text{Dec}((n, d), c) = c^d \bmod n$
- **Correctness:** $\text{Dec}((n, d), \text{Enc}((n, e), m)) = \text{Dec}((n, d), m^e) = m^{e \cdot d} = m^1 = m \bmod n$
 - Since m and n are co-primes, we know that $m^{\phi(n)} = 1 \bmod n$
- **Computational security:** Integer factorisation problem for large primes p and q

Public-Key Encryption

RSA (properties)

- Security:
 - Inverting $\text{Enc}(pk, m)$ is believed to be difficult without knowing sk for a random m
 - Deriving the secret-key is equivalent to factoring $n = p \cdot q$
 - *“The problem of distinguishing prime numbers from composite numbers and of resolving the latter into their prime factors is known to be one of the most important and useful in arithmetic.” (Gauss, 1805)*
- Note that:
 - RSA is deterministic (unlike ElGamal)
 - Encrypting the same message always results in the same ciphertext
 - **Trivial to break security if $0 \leq m < n$ is chosen from a small message space!**
 - **Symmetric cryptography is thousands of times faster for the same security level!**
 - A modulus n of 3072 bits is believed to provide the security of AES-128

Public-Key Encryption

RSA-OAEP

- To mitigate the determinism (as well as mathematical patterns) of RSA and its vulnerability to small messages, it **shall always be used with a padding scheme**
- OAEP (Optimal Asymmetric Encryption Padding) prepares a message M into a stronger message EM to go into RSA
 - Adds randomness to the encoded message
 - Extends message to perfectly fit the modulus range
 - Adds redundancy to the message (for built-in integrity checks)
- The padded RSA-OAEP scheme exhibits very strong security results

